NeuroMEP: The MERN Stack Explained for a FastAPI Developer

Oct 28, 2026 · @mirage

1. How to use this guide and run the app

If you know FastAPI, Pydantic, PostgreSQL and React with TypeScript, you already know about 70 % of NeuroMEP. The frontend is the same React + TypeScript you know; what changes is the backend language (JavaScript/TypeScript on Node.js instead of Python), the web framework (Express instead of FastAPI), the validator (Zod instead of Pydantic) and the database (MongoDB instead of PostgreSQL).

Every section follows the same pattern:

1. **The concept** in plain words.
2. **The Python equivalent** you already know, usually side by side in code.
3. **Where it lives in NeuroMEP**, with the real file path, so you can open it and read along.

Read sections 2 to 7 in order the first time; they build on each other. Sections 8 to 14 can be read when the topic comes up in the WBS. Section 15 is the map of every file, and section 16 is a one-page cheat sheet.

Running NeuroMEP on your computer

You need **Node.js 24** (the project pins it in `.nvmrc`). MongoDB does not need to be installed: in development the API starts its own copy.

```
# go to the project folder
cd C:\Users\HP\Desktop\MAEC_NeuroMEP

# download every library listed in the package.json files (like: pip install
-r requirements.txt)
npm install

# start the API (port 4000) and the web app (port 5173) together, with auto-
reload
npm run dev
```

Open **http://localhost:5173** and sign in as `lead@neuromep.dev` with password `neuromep-dev`. Other development users (`electrical@`, `checker@`, `admin@` … `neuromep.dev`) share that password.

- The first start downloads a MongoDB binary (a few hundred MB) and takes a minute or two.
- Data is kept in `apps/api/.data/`; delete that folder for a clean start.
- Ctrl + C stops both servers. Saving a file reloads the browser automatically.

Command	What it does	Python equivalent
`npm run dev`	API + web with auto-reload	`uvicorn main:app --reload` + `npm run dev`
`npm test`	All automated tests (Vitest)	`pytest`
`npm run typecheck`	Checks types without running	`mypy .`
`npm run lint`	Code-quality and security rules (ESLint)	`ruff check .`
`npm run format`	Auto-format (Prettier)	`black .` / `ruff format`
`npm run build`	Production files in each `dist/`	building a wheel / Docker image
`npm run hash-password -w @neuromep/api`	Hash a password for the user file	a small `passlib` script

2. The big picture: MERN next to your Python stack

MERN is four technologies that together make a web application: **M**ongoDB (database), **E**xpress (web framework), **R**eact (user interface) and **N**ode.js (the runtime that runs JavaScript on the server). NeuroMEP adds **TypeScript** on top of all of them, so every file has types, just like your typed React frontend.

The big difference from your stack: **one language end to end**. In your stack the browser speaks TypeScript and the server speaks Python, so data shapes are defined twice (a Pydantic model and a TS interface). In NeuroMEP both sides are TypeScript, so the data shapes and business rules are written once in `packages/shared` and imported by both.

Job	Your Python stack	NeuroMEP (MERN + TS)	Where in NeuroMEP
Language on the server	Python 3	TypeScript (compiled to JavaScript)	every `.ts` file

Job	Your Python stack	NeuroMEP (MERN + TS)	Where in NeuroMEP
Runtime	CPython + uvicorn (ASGI server)	Node.js 24 (has its own HTTP server)	`apps/api/src/server.ts`
Web framework	FastAPI	Express 5	`apps/api/src/app.ts`
Request/response validation	Pydantic models	Zod schemas	`packages/shared/src/project/schema.ts`
Dependency injection, auth guards	`Depends(...)`	middleware functions	`apps/api/src/auth/middleware.ts`
Database	PostgreSQL (tables, rows, SQL)	MongoDB (collections, JSON documents)	`apps/api/src/projects/repository.ts`
Database access	SQLAlchemy / asyncpg	official `mongodb` driver	same file
Schema migrations	Alembic	migration function on read	`packages/shared/src/project/migrate.ts`
Frontend	React + TS + Vite	React 19 + TS + Vite 8 (same)	`apps/web/`
Package manager	pip / poetry, `requirements.txt` / `pyproject.toml`	npm, `package.json` + `package-lock.json`	root and each package
Tests	pytest, `TestClient`	Vitest, supertest, Testing Library	`*.test.ts(x)` beside each file
Lint / format / types	ruff, black, mypy	ESLint, Prettier, `tsc`	`eslint.config.js`, `.prettierrc.json`, `tsconfig*.json`
Dev auto-reload	`uvicorn --reload`	`tsx watch` (API), Vite (web)	`apps/api/package.json`

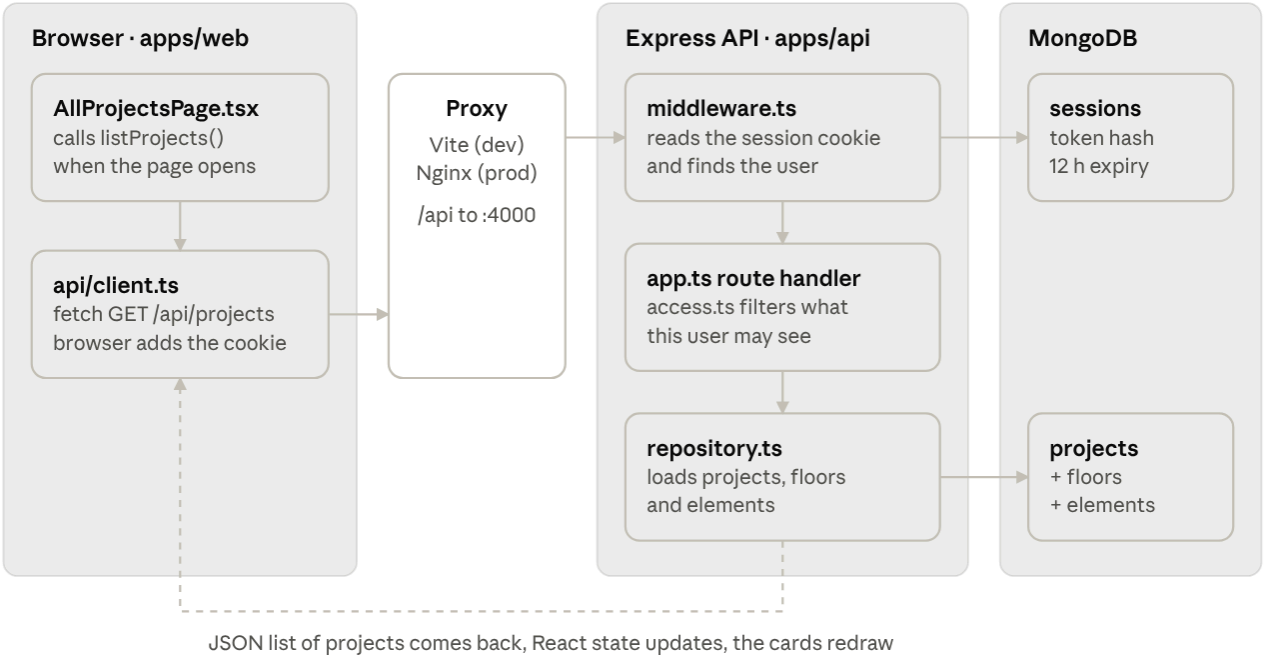
The six layers of NeuroMEP

The design document (Section 7) splits NeuroMEP into six layers. This is how they map to folders:

Layer	Responsibility	Folder
01 Presentation	Screens: Login, All Projects, Import File, Floor Setup, Schematic, Output	`apps/web/src/pages`, `layout`, `viewer`
02 Application controller	Loading, editing, autosaving the open project; navigation rules	`apps/web/src/workspace`
03 Network model	Components, cables, floors as data	`packages/shared/src/model`, `project`
04 Calculation engine	Cable sizing, breakers, checks C1 to C12	`packages/engine` (WBS 1.6)
05 Reference data	Dubai rule pack, CableSizer tables	`packages/rulepacks`
06 Persistence, sync, export	API, MongoDB, files, sessions	`apps/api`

One request, end to end

One request travels from a React page through Express to MongoDB and back



request flow · browser, proxy, Express, MongoDB

Solid arrows are the request going in; the dashed line is the answer coming back. Every box names the file you can open to read that step.

3. The language: TypeScript compared with Python

TypeScript is JavaScript plus type annotations, the same idea as Python type hints. The key fact: **types disappear when the code runs**. `tsc` checks them and then strips them, exactly as mypy checks hints that Python ignores at runtime. That is why NeuroMEP still validates incoming data with Zod (section 8), as FastAPI uses Pydantic.

Browsers and Node.js only run JavaScript. In development `tsx` (API) and Vite (web) strip the types on the fly; `npm run build` writes plain `.js` files into `dist/`.

Everyday syntax side by side

Concept	Python	TypeScript
Variable	`name = 'NeuroMEP'`	`const name = 'NeuroMEP'` (cannot be reassigned) or `let count = 0` (can)
Type hint	`count: int = 0`	`let count: number = 0`
Numbers	`int` and `float`	one `number` type (64-bit float); `bigint` for huge integers
Nothing	`None`	`null` (deliberately empty) and `undefined` (never set)
Text formatting	`f"{floor} has {n} DBs"`	`` `${floor} has ${n} DBs` `` (template literal, backticks)
Dict / object	`{'id': 'f1', 'name': 'GF'}`	`{ id: 'f1', name: 'GF' }`
List / array	`[1, 2, 3]`	`[1, 2, 3]`
Function	`def area(w, h): return w * h`	`function area(w: number, h: number) { return w * h; }`
Lambda	`lambda x: x * 2`	`(x) => x * 2` (arrow function, used everywhere)
Comprehension	`[f.name for f in floors if f.plan]`	`floors.filter((f) => f.plan).map((f) => f.name)`
Unpacking	`a, b = pair` / `**opts`	`const [a, b] = pair` / `...opts` (spread)

Concept	Python	TypeScript
Optional access	`x.plan.calibration if x.plan else None`	`x.plan?.calibration` (optional chaining)
Default if missing	`value if value is not None else 0`	`value ?? 0` (nullish coalescing)
Errors	`try: ... except ValueError as e:`	`try { ... } catch (e) { ... }`
Import	`from app.auth import hash_password`	`import { hashPassword } from './auth/passwords.js'`
Export	everything at module top level is importable	only what is marked `export`

Five traps for a Python developer

1. **Equality.** Always use `===` and `!==`. The double form `==` converts types first (`0 == ''` is true). NeuroMEP's code never uses `==`.
2. **Truthiness differs.** In Python an empty list or dict is falsy; in JavaScript `[]` and `{}` are **truthy**. Check `list.length === 0` instead. `0`, `''`, `null`, `undefined` and `NaN` are falsy in JavaScript (in Python `NaN` is truthy).
3. **`this`.** Inside a class method `this` is like `self`, but it is not passed automatically when a method is handed around as a callback. Arrow functions keep the surrounding `this`, which is why NeuroMEP writes class callbacks as arrows (for example `signIn = vi.fn(async (...) => ...)` in the web tests).
4. **Objects are passed by reference**, as in Python. React and NeuroMEP's reducer rely on never changing an object in place: they build a new one with spread, for example `{ ...p, meta: { ...p.meta, lastPage: a.page } }` in `apps/web/src/workspace/projectReducer.ts`.
5. **Integer maths.** `0.1 + 0.2` is `0.30000000000000004`, the same float behaviour as Python, but there is no separate `int`. Electrical results will be rounded deliberately in the engine.

Types that do real work

Interfaces and type aliases describe shapes, like a `TypedDict` or a Pydantic model without validation:

```
// apps/api/src/auth/sessions.ts
export interface Session {
  userId: string;
  expiresAt: Date;
}
```

Union types say a value is one of several things, like `Literal['setup', 'modelling']` or `Union[A, B]`. A **discriminated union** adds a tag field so TypeScript knows which shape it holds. NeuroMEP uses one for floor plans so a Revit plan can be added later (`kind: 'pdf'` today, `kind: 'revit'` in #TASK-33):

```
if (floor.plan?.kind === 'pdf') {
  // here TypeScript knows floor.plan has page size and calibration
}
```

Generics work like `TypeVar` and `list[T]`: `Collection<SessionDoc>` in `sessions.ts` means a MongoDB collection whose documents have the `SessionDoc` shape.

Strict settings. `tsconfig.base.json` turns on `strict` and `noUncheckedIndexedAccess`. The second means `floors[0]` has type `Floor | undefined`, so the code must handle an empty list. That catches bugs mypy would usually let through.

Classes

Classes look familiar. `private readonly` in the constructor declares and stores a field in one go:

```
// apps/api/src/auth/attempts.ts (shortened)
export class LoginLimiter {
  private readonly failures = new Map<string, number[]>(); // like dict[str,
list[int]]
  constructor(private readonly max: number, private readonly windowMs:
number) {}
  isLocked(key: string): boolean {
    return this.recent(key).length >= this.max;
  }
}
```

The Python version would be a class with `__init__(self, max, window_ms)` and `self.failures: dict[str, list[float]] = {}`. NeuroMEP uses classes for stateful services (session store, limiter, repository) and plain functions everywhere else.

4. Node.js and asynchronous code compared with asyncio

Node.js is a program that runs JavaScript outside the browser. It is Google's V8 engine (the one inside Chrome) plus **libuv**, a C library that does networking, files and timers without blocking. It plays the role CPython + uvicorn play for you: it runs the code and serves HTTP.

One thread, one event loop

Node runs your JavaScript on **one thread** with an **event loop**, the same model as `asyncio`. While one request waits for MongoDB, the loop serves other requests. The difference is that in Node **everything is asynchronous by default**:

	Python + FastAPI	Node.js + Express
Async is	opt-in: `async def`, async libraries (asyncpg, httpx)	the default: the database driver, `fetch`, file APIs all return promises
Blocking by mistake	easy: a sync library inside `async def` freezes the loop	rare: only `...Sync` functions block (NeuroMEP uses `readFileSync` once, at start-up, to read the user file)
Sync endpoints	FastAPI runs `def` endpoints in a thread pool	there are no sync endpoints; every handler is `async`
Start the loop	`asyncio.run(main())`	automatic; top-level `await` works in modules (`apps/api/src/server.ts` awaits the database connection at the top)

Promises compared with coroutines

A **Promise** is a value that will arrive later: like an `asyncio.Future`. `async` / `await` read the same in both languages:

```
// apps/api/src/app.ts (shortened)
app.get('/api/projects', signedIn, async (req, res) => {
  const user = currentUser(req);
  res.json({ projects: await projects.listSummaries((p) =>
canViewProject(user, p)) });
});
```

```
# the FastAPI equivalent
@app.get('/api/projects')
async def list_projects(user: User = Depends(current_user)):
    return {'projects': await repo.list_summaries(lambda p:
can_view_project(user, p))}
```

Three differences matter in practice:

1. **Promises start immediately.** Calling an async function starts it at once; `await` only waits for the result. A Python coroutine does nothing until awaited. So `const a = loadA(); const b = loadB(); await a; await b;` runs both at the same time.
2. **Running in parallel:** `await Promise.all([loadA(), loadB()])` is `await asyncio.gather(load_a(), load_b())`.
3. **A forgotten `await` is silent.** Python warns "coroutine was never awaited"; JavaScript just carries on with a pending promise. When NeuroMEP deliberately does not wait (fire and forget), it writes `void` in front, for example `void editor.save()` in `apps/web/src/workspace/ProjectWorkspace.tsx`, so a reader knows it is intentional.

Callbacks: the older style you will still meet

Before promises, Node APIs took a callback `(err, result) => ...`. NeuroMEP wraps one such API, `scrypt`, in a Promise so it can be awaited:

```
// apps/api/src/auth/passwords.ts
function derive(password: string, salt: Buffer, opts: ScryptOptions):
Promise<Buffer> {
  return new Promise((resolve, reject) =>
    scrypt(password.normalize('NFKC'), salt, KEY_LENGTH, opts, (err, key) =>
      err ? reject(err) : resolve(key),
    ),
  );
}
```

The Python equivalent is `await loop.run_in_executor(None, hashlib.scrypt, ...)`: both push the CPU-heavy hash to a background thread so the loop keeps serving requests.

CPU-heavy work

The event loop must never be busy for long. A cable-sizing run over a whole building would freeze the server or the browser tab if run on the main thread. NeuroMEP's rule (CLAUDE.md, Architecture rules) is that the calculation engine runs in a **Web Worker** in

the browser, a separate thread with its own loop. On the server the equivalent is `worker_threads`. In Python you would reach for `ProcessPoolExecutor` or Celery because of the GIL.

Timers and errors

- `setTimeout(fn, ms)` is `loop.call_later(seconds, fn)`. NeuroMEP's autosave waits for a pause in typing with `setTimeout` and cancels it with `clearTimeout` if you type again (`apps/web/src/workspace/useProjectEditor.ts`).
- A promise that rejects with nobody catching it stops a Node process. Express 5 (the version NeuroMEP uses) catches errors thrown in `async` route handlers and turns them into a 500 response, much like FastAPI's default exception handler.

5. Packages, the monorepo and tooling

npm is Node's pip, and `package.json` is its `pyproject.toml`. NeuroMEP is a **monorepo**: one git repository holding five packages that npm links together as "workspaces", so the API and the web app import shared code as if it were an installed library.

package.json, piece by piece

```
{
  "name": "@neuromep/web",            // package name; @neuromep is the
"scope"
  "type": "module",                   // use modern import/export (ES
modules)
  "scripts": { "dev": "vite", "test": "vitest run" },   // npm run dev, npm
test
  "dependencies": {                   // needed at run time (like
install_requires)
    "@neuromep/shared": "0.1.0",       // a workspace package, linked, not
downloaded
    "react": "^19.3.1"                 // ^ = any 19.x.y at least 19.3.1
  },
  "devDependencies": {                // only for development and tests (like
dev extras)
    "@testing-library/react": "^16.3.3"
  }
}
```

(`apps/web/package.json`, comments added; real JSON allows no comments.)

Python	Node / npm	Notes
`venv` per project, activated	`node_modules/` per project, nothing to activate	Node looks for `node_modules` next to the code automatically
`pip install -r requirements.txt`	`npm install`	installs from `package.json` + `package-lock.json`
`requirements.txt` with `==` pins / `poetry.lock`	`package-lock.json`	exact versions of every library, committed, never edited by hand
`pip install x`	`npm install x` (or `-D` for dev)	adds it to `package.json`
`python -m tool` / `pipx run`	`npx tool`	runs a tool from `node_modules/.bin`
`pip-audit`	`npm audit`	known security advisories; CI fails on "high"
`.python-version` (pyenv)	`.nvmrc`	pins Node 24

Workspaces: how the five packages find each other

The root `package.json` lists the workspaces: `packages/shared`, `packages/rulepacks`, `packages/engine`, `apps/api`, `apps/web`. `npm install` creates links in `node_modules/@neuromep/` pointing at those folders. `-w` runs a script in one package: `npm run hash-password -w @neuromep/api`.

Each internal package declares what it exposes in `exports`:

```
// packages/shared/package.json
"exports": {
  ".": { "source": "./src/index.ts", "types": "./dist/index.d.ts", "default": "./dist/index.js" },
  "./testing": { "source": "./src/testing.ts", ... }
}
```

The `source` condition is a NeuroMEP convention: in development, tests and type checks, tools read the TypeScript source directly (no build step); in production Node loads the compiled `dist/index.js`. In Python terms it is an editable install (`pip install -e .`) for development and a built wheel for production.

Modules: import and export

- `"type": "module"` means ES modules: `import { x } from './file.js'`. The older CommonJS style (`require()`, `module.exports`) appears in some libraries and in the scratch scripts, but not in NeuroMEP's code.
- Imports inside the API and packages end in `.js` even though the file is `.ts`. This is the Node rule (`"moduleResolution": "NodeNext"` in `tsconfig.base.json`): the import names the file that will exist after compiling. The web app, bundled by Vite, leaves the extension off.
- Only names marked `export` are importable; an `index.ts` re-exports a folder, like a package `__init__.py` (`packages/shared/src/index.ts`).

The tools around the code

Job	NeuroMEP	Python equivalent	Config file
Type check	`tsc` (TypeScript compiler)	mypy / pyright	`tsconfig.base.json` + one per package
Run TS directly in development	`tsx watch` restarts the API on save	`uvicorn --reload`	`apps/api/package.json`
Frontend dev server and bundler	Vite: hot reload, `/api` proxy, production bundle	(Vite too, in your React project)	`apps/web/vite.config.ts`
Lint	ESLint + typescript-eslint + react-hooks + security plugin	ruff / flake8 + bandit	`eslint.config.js`
Format	Prettier	black / ruff format	`.prettierrc.json`
Environment variables	`--env-file-if-exists=../../.env`, read and validated with Zod	python-dotenv / pydantic-settings	`.env.example`, `apps/api/src/config.ts`
Security scan	Semgrep in CI	bandit / Semgrep	`.github/workflows/ci.yml`

Each package has two TypeScript configs: `tsconfig.json` (type check including tests) and `tsconfig.build.json` (compile `src` into `dist`, tests excluded).

6. HTTP and REST, with NeuroMEP's real endpoints

HTTP does not change between Python and Node: a request is a method, a path, headers and an optional body; a response is a status code, headers and an optional body. Everything you know from FastAPI applies; only the code that reads and writes these parts looks different.

Anatomy of one NeuroMEP request

```
PUT /api/projects/3f2a9c1e HTTP/1.1          <- method + path (the project id
is a path parameter)
Host: localhost:5173
Content-Type: application/json              <- the body is JSON
Cookie: neuromep_session=Yk3n...            <- the session cookie, sent by
the browser automatically

{"project": {"id": "3f2a9c1e", "version": 7, "details": {...}, ...}}
```

```
HTTP/1.1 409 Conflict                       <- someone else saved version 8
meanwhile
Content-Type: application/json

{"code": "E0802", "error": "...", "currentVersion": 8}
```

REST: URLs name things, methods say what to do

REST is a convention, not a library: a URL names a resource (a project, a plan file) and the HTTP method says what to do with it. These are all of NeuroMEP's endpoints today (`apps/api/src/app.ts`):

Method + path	Purpose	Success	Typical errors
`GET /api/health`	Is the server up? (used by monitoring)	200	
`POST /api/auth/sign-in`	Check email + password, start a session, set the cookie	200 `{user}`	401 E0101, 429 E0103
`POST /api/auth/sign-out`	End the session, clear the cookie	204 (no body)	

Method + path	Purpose	Success	Typical errors
`GET /api/auth/me`	Who is signed in? (on page load)	200 `{user}`	401 E0102
`GET /api/users`	Users for the engineer pickers	200 `{users}`	401
`GET /api/projects`	Projects this user may see (All Projects)	200 `{projects}`	401
`POST /api/projects`	New project, or import a `.json` project	201 `{project}`	403 E0202, 400 E0203
`GET /api/projects/:id`	Load one project	200 `{project}`	404 E0201
`PUT /api/projects/:id`	Save the whole project (optimistic locking)	200 `{project}`	403 E0301, 400 E0803, 409 E0802
`POST /api/projects/:id/plans`	Upload a floor-plan PDF (raw bytes, not JSON)	201 `{fileId, fileName, uploadedAt}`	400 E0602
`GET /api/projects/:id/plans/:fileId`	Download the PDF (streamed)	200 PDF bytes	404 E0605

Methods and status codes in practice

- **GET** reads and never changes anything; it can be repeated safely. **POST** creates or performs an action. **PUT** replaces a whole resource (NeuroMEP sends the whole project on save). **DELETE** removes; none exists yet. **PATCH** (partial update) is not used.
- **2xx** success: 200 OK, 201 Created (a new project or plan), 204 No Content (sign-out).
- **4xx** the client must change something: 400 invalid data, 401 not signed in, 403 signed in but not allowed (an engineer creating a project), 404 not found **or not visible to you** (NeuroMEP never reveals that a project exists if you cannot see it), 409 version conflict, 429 too many attempts.
- **5xx** a server bug or outage.

Every NeuroMEP error body has the same shape, `{code, error}`, from the catalogue in `packages/shared/src/errors/catalogue.ts`. FastAPI's equivalent is raising `HTTPException(status_code=409, detail={...})`.

Headers and bodies you will meet

- `Content-Type` says what the body is: `application/json` for data, `application/pdf` for plan uploads. FastAPI's `UploadFile` usually means `multipart/form-data`; NeuroMEP sends the PDF as the raw body and the file name in an `X-File-Name` header, which is simpler for one file.
- `Cookie` (browser to server) and `Set-Cookie` (server to browser) carry the session (section 9).
- Large downloads are **streamed**: `stream.pipe(res)` sends the PDF in chunks without loading it into memory, like FastAPI's `StreamingResponse`.

Calling the API from the browser

The browser's built-in `fetch` is the client (your equivalent may be `axios`, or `httpx` / `requests` in Python). NeuroMEP wraps it in one class, `ApiClient` in `apps/web/src/api/client.ts`, with one method per endpoint, so pages call `api.saveProject(project)` and never write URLs by hand.

Same origin, proxies and CORS

A browser only sends cookies to, and lets pages read responses from, the **same origin** (scheme + host + port) unless the server allows otherwise with **CORS** headers, which FastAPI handles with `CORSMiddleware`. NeuroMEP avoids CORS entirely: in development the Vite server on port 5173 forwards every `/api/...` request to the API on 4000 (`proxy` in `apps/web/vite.config.ts`); in production Nginx serves the web files and forwards `/api` to Node on the same domain. The browser sees one site.

7. Express compared with FastAPI

Express is smaller than FastAPI: it routes requests and runs **middleware**, and that is all. It does not validate, does not read type hints and does not generate `/docs`. NeuroMEP adds validation with Zod and keeps error answers uniform with `sendError`. In exchange Express is simple to read: a request passes through a list of functions, in order.

The same endpoint in both frameworks

Saving a project, `PUT /api/projects/:id` (`apps/api/src/app.ts`, shortened):

```
app.put('/api/projects/:id', signedIn, async (req, res) => {
  const user = currentUser(req);                        // set by the
signedIn middleware
  if (!(await loadVisible(req, res))) return;           // 404 if missing or
not visible
  if (!canEditProjectSetup(user)) return sendError(res, 403, 'E0301');
  const parsed = ProjectSchema.safeParse(req.body?.project);   // validate
the body (Zod)
  if (!parsed.success || parsed.data.id !== req.params.id) return
sendError(res, 400, 'E0803');
  const result = await projects.save(parsed.data, now());
  if (result.ok) res.json({ project: result.project });
  else if (result.reason === 'conflict')
    sendError(res, 409, 'E0802', { currentVersion: result.currentVersion });
  else sendError(res, 404, 'E0201');
});
```

```
@app.put('/api/projects/{id}')
async def save_project(id: str, body: SaveBody, user: User =
Depends(signed_in)):
    await load_visible(id, user)                          # raises
HTTPException(404)
    if not can_edit_project_setup(user):
        raise HTTPException(403, {'code': 'E0301', ...})
    if body.project.id != id:                             # body already
validated by Pydantic
        raise HTTPException(400, {'code': 'E0803', ...})
    result = await projects.save(body.project, now())
    ...
    return {'project': result.project}
```

FastAPI	Express	In NeuroMEP
`app = FastAPI()`	`const app = express()`	`createApp()` in `app.ts`
`uvicorn main:app`	`app.listen(4000)`	`server.ts`
`@app.put('/x/{id}')` decorator	`app.put('/x/:id', ...handlers)`	`app.ts`
path parameter as a typed argument `id: str`	`req.params.id` (always a string)	`req.params.id`
query parameter `q: str = None`	`req.query.q`	not used yet
body parsed and validated from a Pydantic model	`express.json()` parses; you validate with Zod	`ProjectSchema.safeParse(req.body?.project)`
`return {...}` is the response	call `res.json({...})` / `res.status(201).json(...)`	every route

FastAPI	Express	In NeuroMEP
`raise HTTPException(...)`	`sendError(res, status, code)` then `return`	`apps/api/src/errors.ts`
`Depends(get_current_user)`	a middleware placed before the handler	`signedIn` = `requireUser(auth, sessions)`
`@app.middleware('http')`	`app.use(fn)`	`app.use(express.json({ limit: '20mb' }))`
`app.dependency_overrides` in tests	pass different objects into `createApp(deps)`	tests pass an in-memory session store and a test clock
automatic OpenAPI `/docs`	none (could be added later)	the endpoint table in section 6

Middleware: the heart of Express

A middleware is a function `(req, res, next)`. It may answer the request itself (and stop), or call `next()` to pass it on. The route's `signedIn` middleware is NeuroMEP's equivalent of `Depends(get_current_user)`:

```
// apps/api/src/auth/middleware.ts
export function requireUser(auth: AuthProvider, sessions: SessionStore) {
  return async (req: Request, res: Response, next: NextFunction) => {
    const token = sessionToken(req);                       // read the cookie
    const session = token ? await sessions.get(token) : null;
    const user = session ? await auth.findUser(session.userId) : null;
    if (!user) return sendError(res, 401, 'E0102');        // stop here: not
signed in
    req.user = user;                                       // hand the user
to the next function
    next();
  };
}
```

In FastAPI the dependency **returns** the user and FastAPI injects it as an argument. In Express the middleware **attaches** it to `req`, and the handler reads it back with `currentUser(req)`. TypeScript is told that `req.user` exists through a small "declaration merging" block at the top of `middleware.ts`.

The life of a request in NeuroMEP's API

1. Node receives the bytes; Express builds `req` and `res`.
2. `express.json()` parses a JSON body into `req.body` (or `express.raw()` keeps PDF bytes as a Buffer on the upload route).
3. Express finds the first route whose method and path match.
4. Route middleware runs: `signedIn` checks the session.

5. The handler checks permissions (`packages/shared/src/access.ts`), validates with Zod, calls the repository, and answers with `res.json` or `sendError` .
6. If anything throws, Express 5 catches it and answers 500.

Dependency injection without a framework

`createApp(deps)` receives everything it needs (auth provider, session store, repository, plan storage, clock, id generator) instead of creating them. `server.ts` passes real MongoDB-backed objects; the tests pass a temporary database and a controllable clock to test 12-hour session expiry in milliseconds. This is the plain-code version of FastAPI's `Depends` + `dependency_overrides` .

8. Validation and shared types: Zod compared with Pydantic

Zod plays Pydantic's role. Because TypeScript types vanish at run time (section 3), data arriving from outside (a request body, a `.json` project file, the user file, environment variables) must be checked by code. With Pydantic you write a class and get both the type and the validator. With Zod you write a **schema value** and derive the TypeScript type from it with `z.infer` , so you also write the shape once.

The user file, both ways

```
// apps/api/src/auth/config-provider.ts
const StoredUserSchema = z.object({
  id: z.string().min(1),
  name: z.string().min(1),
  email: z.email(),
  role: RoleSchema,                                    //
z.enum(['TECHNICAL_LEAD', ...])
  passwordHash: z.string().startsWith('scrypt$'),
});
export type StoredUser = z.infer<typeof StoredUserSchema>;   // the TS type,
for free

const parsed = UsersFileSchema.safeParse(raw);
if (!parsed.success) throw new Error(z.prettifyError(parsed.error));
```

```
class StoredUser(BaseModel):
    id: str = Field(min_length=1)
    name: str = Field(min_length=1)
    email: EmailStr
    role: Role                                          # an Enum
    password_hash: str = Field(pattern=r'^scrypt\$')

try:
    users = UsersFile.model_validate(raw)
except ValidationError as e:
    raise RuntimeError(str(e))
```

Pydantic	Zod	NeuroMEP example
`class X(BaseModel)`	`const XSchema = z.object({...})` + `type X = z.infer<typeof XSchema>`	`ProjectSchema` in `packages/shared/src/project/schema.ts`
`X.model_validate(data)` raises	`XSchema.parse(data)` throws	rule packs loaded with `RulePackSchema.parse`
try/except `ValidationError`	`XSchema.safeParse(data)` returns `{success, data}` or `{success, error}`	every API body
`Literal['a','b']` / `Enum`	`z.enum(['a','b'])`	roles, design stages, page keys
`Optional[int] = None`	`z.number().nullable()` (null) / `.optional()` (missing)	demand factor `null` = TBC
`model_config = ConfigDict(extra='forbid')`	`z.strictObject({...})`	rule-pack schema
`@field_validator` / `@model_validator`	`.refine(fn, message)`	unique emails in the user file
`Field(discriminator='kind')`	`z.discriminatedUnion('kind', [...])`	`network elements by kind: ElementSchema in model/elements.ts`
coercion by default (`'5'` to `5`)	no coercion unless you ask: `z.coerce.number()`	`API_PORT` in `apps/api/src/config.ts`
`BaseSettings` for env vars	a Zod schema over `process.env`	`loadConfig()` in `config.ts`

One behaviour to remember: **Pydantic converts, Zod checks.** Pydantic quietly turns `"5"` into `5`; Zod rejects it unless the schema says `z.coerce`. NeuroMEP prefers that strictness for engineering data.

Why `packages/shared` matters

In your stack the Pydantic model on the server and the TypeScript interface on the client can drift apart. In NeuroMEP both sides import the **same** schema and the same rules:

- The web app uses `page1Issues()` from `readiness.ts` to show the red field messages and to decide whether Next works.

- The API uses `ProjectSchema` to reject a malformed save, and `access.ts` to decide who may see or change a project. The browser uses `access.ts` too, to grey out what you cannot do.
- The engine will import the model types from `packages/shared/src/model`.

The browser's checks are for the user's convenience; the server's checks are the real gate, because anyone can send a request without the browser.

Changing the shape of stored data

PostgreSQL changes need an Alembic migration that alters tables. MongoDB stores documents without a fixed schema, so NeuroMEP handles shape changes in code:

- Every project carries `schemaVersion` (2 today). `migrateProjectData()` in `packages/shared/src/project/migrate.ts` upgrades an older document **when it is read**, and the next save stores the new shape.
- A new **optional** field needs no migration at all. `meta.reachedPage` (tab order, A-12) was added as `PageKeySchema.optional()`, and `reachedPage()` in `progress.ts` treats a missing value as "the last page opened".

9. Authentication and security

NeuroMEP uses **server-side sessions in an httpOnly cookie**, not the JWT-in-localStorage pattern common in FastAPI tutorials. Both are valid; this section explains each piece and why NeuroMEP chose sessions (WBS 1.2.2).

Authentication compared with authorisation

- **Authentication**: who are you? Email and password, then a session. Failure is **401**.
- **Authorisation**: may you do this? Decided by role and project assignment in `packages/shared/src/access.ts`. Failure is **403**, or **404** when even the project's existence must stay hidden.

Passwords: never stored, only hashed

The user file (`apps/api/config/users.dev.json` in development, `USERS_FILE` in production) stores a **hash** such as `scrypt$16384$8$1$<salt>$<hash>`. A hash is a one-way function: the server hashes the password typed at sign-in and compares the results.

- **Salt**: 16 random bytes per user, so two users with the same password get different hashes, and pre-computed tables are useless.

- **Slow on purpose**: scrypt costs about 16 MB of memory and tens of milliseconds per try, which makes guessing millions of passwords impractical. Python's `passlib` with bcrypt or `argon2-cffi` serve the same purpose; NeuroMEP uses Node's built-in `crypto.scrypt`, so no extra library is needed.
- **Constant-time comparison** (`timingSafeEqual`) and a **dummy hash for unknown emails** make a wrong email and a wrong password take the same time and give the same message (E0101), so an attacker cannot discover which emails exist.

Sessions compared with JWT

	JWT in `Authorization: Bearer` (typical FastAPI)	Session cookie (NeuroMEP)
What the browser holds	a signed token containing the user id and expiry	a random 43-character token that means nothing on its own
Where it is kept	usually `localStorage`, read by JavaScript	an **httpOnly** cookie that JavaScript cannot read
How the server checks it	verifies the signature, no database call	looks up the token's hash in the `sessions` collection
Sign out	the token stays valid until it expires	the session is deleted; the token is dead at once
Risk if the page has an XSS bug	the script can steal the token and use it elsewhere	the script cannot read the cookie
Cost	none per request	one indexed database read per request

NeuroMEP stores only the **SHA-256 of the token** in MongoDB, so even a copy of the database cannot be replayed as a login. MongoDB deletes expired sessions by itself through a **TTL index** on `expiresAt` (12 hours). Code: `apps/api/src/auth/sessions.ts`.

The cookie, attribute by attribute

```
Set-Cookie: neuromep_session=Yk3n...; Path=/; HttpOnly; SameSite=Strict;
Secure; Expires=Wed, 29 Oct 2026 10:00:00 GMT
```

Attribute	Meaning
`HttpOnly`	page JavaScript cannot read it (`document.cookie` is empty); only sent with requests

Attribute	Meaning
`SameSite=Strict`	not sent on requests started by another website, which blocks cross-site request forgery (CSRF)
`Secure`	only sent over HTTPS; on in production, off on `http://localhost`
`Path=/`	sent with every request to the site
`Expires`	when the browser drops it; the server's own 12-hour expiry is what really counts

The browser attaches the cookie automatically; `apps/web/src/api/client.ts` only sets `credentials: 'same-origin'`. The web code never sees a token at all. When any data request gets a 401 (session expired, signed out on another tab), the client calls a handler that clears the user, and the router sends you to Login (`apps/web/src/auth/AuthContext.tsx`).

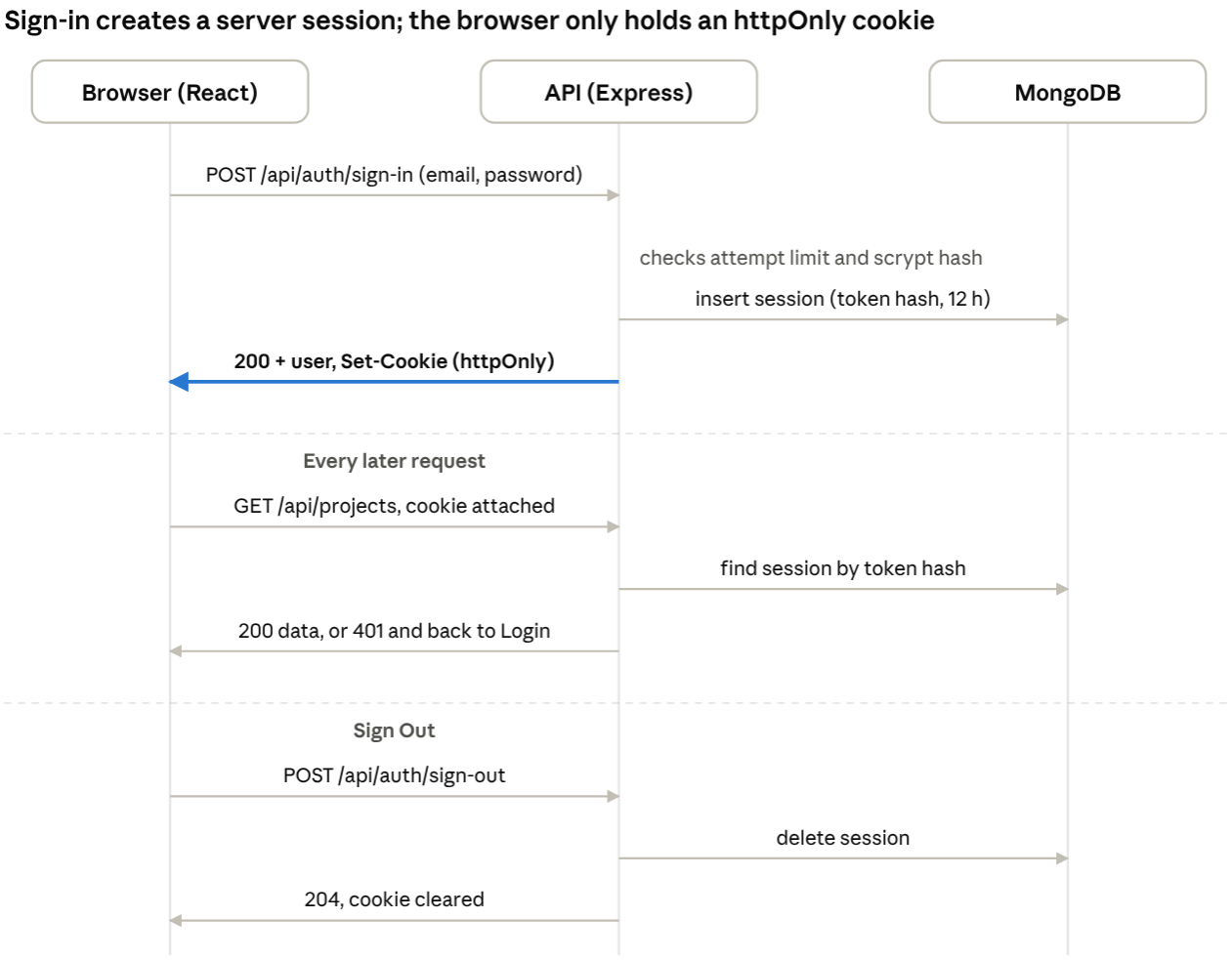
Login attempt limit

`LoginLimiter` (`apps/api/src/auth/attempts.ts`) counts failures per email (5 in 15 minutes) and per client address (20 in 15 minutes); the next attempt gets **429 E0103**. It lives in memory, like a simple `slowapi` limiter in FastAPI. Behind Nginx, `app.set('trust proxy', 'loopback')` makes Express read the real client address from `X-Forwarded-For`.

Built to be replaced

The company login (MAEC One, open item OI-28) may later be single sign-on with OpenID Connect. Everything about **who** the users are sits behind the `AuthProvider` interface (`apps/api/src/auth/provider.ts`: `authenticate`, `findUser`, `listUsers`). Replacing the user file with MAEC One means writing one new class; sessions, cookies, middleware and the web app stay the same.

The sign-in sequence



sign-in sequence · browser, API, MongoDB

The highlighted answer is the only moment the token travels to the browser, and page JavaScript can never read it; every later request is checked against MongoDB.

10. MongoDB compared with PostgreSQL

MongoDB stores **documents** (JSON-like objects, saved in a binary form called BSON) in **collections**, instead of rows in tables. A document can hold nested objects and arrays, so a whole NeuroMEP project, with its details, site conditions, demand factors and disciplines, is naturally one object, exactly the shape the React app and Zod already use. There is no ORM and no SQL: the official `mongodb` driver takes JavaScript objects as queries.

Vocabulary

PostgreSQL	MongoDB	NeuroMEP
database	database	`neuromep`
table	collection	`projects`, `floors`, `elements`, `auditLog`, `sessions` (and GridFS for plans)
row	document	one project, one floor, one session
primary key `id`	`_id` (any unique value)	the project's UUID; `projectId/floorId` for floors
columns fixed by `CREATE TABLE`	fields free per document; NeuroMEP fixes them with Zod	`ProjectSchema`
foreign key + JOIN	a stored id + a second query (or `$lookup`)	`floors.projectId`; the repository reassembles
`CREATE INDEX`	`createIndex`	`floors.createIndex({ projectId: 1 })` in `repository.ts`
SQLAlchemy / asyncpg	the `mongodb` driver	`Collection<FloorDoc>` typed with TypeScript
Alembic migration	migration in code on read	`migrate.ts` (section 8)
`BYTEA` / large objects / S3	GridFS (files split into chunks)	floor-plan PDFs today; server disk in WBS 1.2.7 (A-02)

Queries side by side

```
// MongoDB driver (TypeScript)
await floors.find({ projectId: pid }).toArray();
await floors.deleteMany({ projectId: pid, _id: { $nin: keepIds } });
await sessions.findOne({ _id: digest(token) });
```

```
-- PostgreSQL equivalents
SELECT * FROM floors WHERE project_id = :pid;
DELETE FROM floors WHERE project_id = :pid AND id <> ALL(:keep_ids);
SELECT * FROM sessions WHERE id = :digest;
```

Operators start with `$`: `$set` (update fields), `$in` / `$nin` (in / not in a list), `$inc` (add to a number), `$setOnInsert` (only when creating).

How a project is stored

One project is split over four collections so that large parts can grow independently, then reassembled by `ProjectRepository.assemble()` on read:

- `projects`: details, site, calculation set-up, disciplines, meta, and the list of floor ids.
- `floors`: one document per floor, including its plan reference and calibration.
- `elements`: every network element and cable of Floor Setup, one document each.
- `auditLog`: append-only change history (written with `$setOnInsert`, so an entry is never rewritten).

In PostgreSQL you would design the same four tables with foreign keys. The difference is that each NeuroMEP document keeps its nested parts (a floor's calibration, an element's ratings) inside itself, where PostgreSQL would need extra tables or a JSONB column.

Transactions and the replica set

Saving a project changes several collections, which must succeed or fail together. MongoDB supports multi-document **transactions**, but only on a **replica set** (a group of servers holding copies of the data; one node is enough). That is why development starts a one-node replica set (`startDevMongo` in `apps/api/src/db/mongo.ts`) and production must run MongoDB as a replica set too.

```
await this.client.withSession((session) =>
  session.withTransaction(async () => {
    // every write in here passes { session } and commits together, or not at
all
  }),
);
```

That is SQLAlchemy's `async with session.begin():` in MongoDB form.

Optimistic locking with a version number

Two engineers may open the same project. Each save sends the `version` it was loaded at, and the update only matches if nobody has saved since:

```
// apps/api/src/projects/repository.ts (shortened)
const updated = await this.projects.findOneAndUpdate(
  { _id, version: expected },          // match id AND the version I started
from
  { $set: doc },                       // doc carries version: expected + 1
  { session, returnDocument: 'after' },
);
if (!updated) { /* someone saved first: answer 409 E0802 with their version
*/ }
```

In SQL this is `UPDATE projects SET ..., version = version + 1 WHERE id = :id AND version = :expected` and checking that one row changed. The web app shows the conflict and offers to reload.

Indexes and TTL

Indexes speed up lookups exactly as in PostgreSQL. MongoDB adds **TTL indexes**: `sessions.createIndex({ expiresAt: 1 }, { expireAfterSeconds: 0 })` makes the database delete each session when its `expiresAt` passes; in PostgreSQL you would need a scheduled cleanup job.

When PostgreSQL would have been the better fit

MongoDB suits NeuroMEP because projects are nested documents that are loaded and saved whole. Heavy cross-project reporting with many JOINs, or strict relational integrity enforced by the database, would favour PostgreSQL. In NeuroMEP that integrity is enforced in code by Zod and the repository.

11. React in NeuroMEP

The frontend is the React + TypeScript + Vite you already use, at React 19 and React Router 7. This section skips the basics and covers the patterns NeuroMEP relies on, each with its file.

Start-up and routing

`apps/web/src/main.tsx` creates the `ApiClient` and the `HttpAuthProvider`, then renders `<App>`. `apps/web/src/App.tsx` declares the routes as nested objects (a React Router "data router"):

URL	Page	Notes
`/login`	`LoginPage`	open to everyone

URL	Page	Notes
`/projects`	`AllProjectsPage`	inside `ProtectedRoute`
`/projects/new`	`NewProjectPage`	Technical Lead only
`/projects/:projectId/import`	`ImportFilePage`	inside `ProjectWorkspace`
`/projects/:projectId/floor`	`FloorSetupPage` (`?tab=building` shows the Building tab)	
`/projects/:projectId/schematic`	`SchematicPage`	
`/projects/:projectId/output`	`OutputPage`	

A parent route renders `<Outlet />` where its child page goes. `ProtectedRoute` either renders `<Outlet />` or `<Navigate to="/login" />`. `ProjectWorkspace` loads the project once and renders the four project tabs through its `<Outlet />`, so switching tabs never reloads the project.

Context: data many components need

- `AuthContext` (`apps/web/src/auth/AuthContext.tsx`) holds the signed-in user and the API client. Any component calls `useUser()` or `useApi()`.
- `WorkspaceContext` (`apps/web/src/workspace/ProjectWorkspace.tsx`) holds the open project, its editor and the selected floor: `useWorkspace()`.
- Small contexts in `Shell.tsx` carry the notice toast (`useNotice()`) and the furthest tab reached (`ReachedTabContext`).

Think of context as FastAPI's dependency injection for the component tree: provided once near the top, requested where needed.

The reducer: every change has a name

All edits to a project go through one pure function, `projectReducer(project, action)` in `apps/web/src/workspace/projectReducer.ts`. An action is a small object such as `{ type: 'demandFactor', table: 'byLoadType', key: '1', value: 0.8, audit }`. The reducer returns a **new** project object (spread copies, never in-place edits) and appends an audit entry. Benefits: every possible change is listed in one file, it is tested without a browser, and React detects changes by comparing object identity.

A custom hook that owns saving

`useProjectEditor(projectId)` (`apps/web/src/workspace/useProjectEditor.ts`) is the application controller:

- loads the project, exposes `dispatch(action)` (which runs the reducer) and `save()`;
- **autosaves** a moment after the last change, using `setTimeout` / `clearTimeout` (WBS 1.2.5 sets this to every 10 s);
- warns before the browser tab closes with unsaved changes (`beforeunload`);
- uses React Router's `useBlocker` to save before you move to another page, and asks if the save fails;
- reports a 409 version conflict so the page can offer "Reload".

Effects, refs and the React Compiler rules

- `useEffect` runs after rendering, for things outside React: timers, listeners, redirects. It returns a cleanup function, for example `return () => clearTimeout(t)`. In development, `StrictMode` runs effects twice on purpose to expose missing cleanups.
- `useRef` holds a value that survives re-renders without causing one (the hidden file input on Floor Setup, the latest project inside the editor).
- `useImperativeHandle` lets a parent call methods on a child: the toolbar's Zoom In calls `viewer.current.zoomIn()` on `PlanViewer`.
- The linter applies the React Compiler rules (`eslint-plugin-react-hooks` v7): components must be pure, state is not set directly inside an effect, refs are not read during rendering. Values that can be computed from props are computed while rendering, not copied into state.

Forms and inputs

`NumberInput` (`apps/web/src/components/fields.tsx`) keeps what you type as text and converts it to a number only when valid. With `commit="blur"` it reports the value when you leave the field, so one edit makes one audit entry. Field errors come from the shared `page1Issues()` (section 8) and appear once a field has been touched, or all at once when you press Save or Next.

Loading heavy code only when needed

PDF.js is large, so `apps/web/src/viewer/pdf.ts` loads it with a dynamic `import()` the first time a plan is shown. Vite splits it into its own file, and the login page stays light. Python's closest analogue is importing a heavy module inside a function.

Styling

Plain CSS files sit next to the components that use them (`floor.css` , `shell.css`). Colours are CSS variables defined once in `apps/web/src/styles/global.css` ; the dark theme swaps the variables, which is how the Appearance toggle works.

12. Testing

NeuroMEP has 154 automated tests that run with `npm test` in about 30 seconds. They use **Vitest**, which plays pytest's role, and sit next to the file they test (`passwords.ts` and `auth.test.ts` in the same folder). One root `vitest.config.ts` runs the five packages as separate "projects".

pytest world	NeuroMEP	Example
`def test_x(): assert a == b`	`it('...', () => { expect(a).toBe(b) })` inside `describe('...')`	every `*.test.ts`
fixtures, `@pytest.fixture`	`beforeAll` , `beforeEach` , `afterAll`	a fresh database before each API test
`unittest.mock.Mock` , `monkeypatch`	`vi.fn()` , `vi.stubGlobal('fetch', ...)`	`apps/web/src/auth/auth.test.ts`
FastAPI `TestClient`	`supertest` : `request(app).post(...).send(...)`	`apps/api/src/api.test.ts`
testcontainers / pytest-postgresql	`mongodb-memory-server` : a real MongoDB started for the tests	`api.test.ts`
freezegun	inject a clock: `new MongoSessionStore(db, () => clock)`	session expiry test
pytest-cov	`npm run coverage` (V8 coverage); engine must stay at 90 % or more	`packages/engine`

An API test, both ways

```
// apps/api/src/api.test.ts
it('locks an email for 15 minutes after 5 failed sign-ins (E0103)', async ()
=> {
  for (let i = 0; i < 5; i++) expect((await signIn(LEAD,
'wrong')).status).toBe(401);
  const locked = await signIn(LEAD);
  expect(locked.status).toBe(429);
  expect(locked.body.code).toBe('E0103');
  expect((await signIn(ELEC)).status).toBe(200);   // other users are not
affected
  clock += 15 * 60_000 + 1;                         // move the test clock
past the window
  expect((await signIn(LEAD)).status).toBe(200);
});
```

```
def test_locks_email_after_5_failures(client, clock):
    for _ in range(5):
        assert sign_in(client, LEAD, 'wrong').status_code == 401
    locked = sign_in(client, LEAD)
    assert locked.status_code == 429 and locked.json()['code'] == 'E0103'
    assert sign_in(client, ELEC).status_code == 200
    clock.advance(minutes=15, milliseconds=1)
    assert sign_in(client, LEAD).status_code == 200
```

Screen tests with Testing Library

`apps/web/src/app.test.tsx` renders the whole app in **jsdom** (a simulated browser) and uses it the way a person would: find things by their role and visible name, then click and type.

```
renderAt('/projects/p-1/import', new FakeAuth(LEAD),
fakeApi(completeProject()));
await userEvent.click(tool('Next'));                                  // the
toolbar's Next button
await screen.findByText('TRANSFORMER CAPACITY & CONNECTED LOAD SCHEDULE');
// wait for Floor Setup
```

- `getBy...` finds now or fails; `findBy...` **waits** (for data loading or a page change); `queryBy...` returns null instead of failing, for "is not there" checks.
- Querying by role and label (`getByRole('button', { name: 'Save' })`) tests what the user sees and doubles as an accessibility check.

- `FakeAuth` and `fakeApi` replace the real network, the same way you would override a dependency in FastAPI tests.
- A lesson from building the tab-order test: after a navigation, wait for something on the **new** page before clicking again; checking only the URL can click the old page while React is still swapping it.

Tests that guard the architecture and the data

- `packages/engine/src/boundary.test.ts` fails if the engine ever depends on anything besides `shared`, `rulepacks` and `zod`.
- `packages/rulepacks/src/cablesizer/equivalence.test.ts` proves that every converted CableSizer table row returns the same value as the original Excel rule.
- `packages/shared/src/errors/catalogue.test.ts` checks every error code follows the `E` + task + sequence format and appears once.

Test names carry their #TASK code, for example `describe('#TASK-01 authentication (WBS 1.2.2)')`, so a failing test points straight at the feature.

Later: end-to-end tests

WBS 1.5.2 adds **Playwright**, which drives a real browser against the running app (the Python version is `pytest-playwright`). It was used informally already to compare the screens with the Figma images and to check the session cookie in Edge.

13. Build, CI and deployment

In production NeuroMEP is two things: a folder of **static files** (the React app) served by Nginx, and **one Node process** (the API) that Nginx forwards `/api` requests to. MongoDB runs beside them. The setup itself is WBS 1.8 (staging demo on 10 Nov 2026); this section explains the pieces in advance.

What `npm run build` produces

Package	Output	Runs as
`apps/web`	`apps/web/dist/`: `index.html` + JavaScript and CSS files with content hashes in their names (`index-3fa9c1.js`)	static files; the browser downloads them once and caches them
`apps/api`	`apps/api/dist/*.js` (compiled from `src`)	`node dist/server.js` (the `start` script)

Package	Output	Runs as
`packages/*`	`dist/*.js` + `*.d.ts` type files	loaded by the API in production

Compared with Python: the API build is like compiling a wheel, except TypeScript must be compiled before Node runs it. The web build is identical to your React projects.

Production configuration

The API reads its settings from environment variables, validated by Zod in `apps/api/src/config.ts`, and refuses to start if something required is missing:

Variable	Production value
`NODE_ENV`	`production` (turns on the Secure cookie and trusted proxy)
`MONGODB_URI`	connection string of the replica set (VPS or Atlas)
`USERS_FILE`	path to the user file, for example `/etc/neuromep/users.json`, readable only by the API's account
`API_PORT`	4000 (only Nginx talks to it)

Install on a server with `npm ci`, the strict version of `npm install` that only follows `package-lock.json` (like `pip install --require-hashes`).

Running the API: PM2 or Docker

- **PM2** is a process manager for Node: it starts `node dist/server.js`, restarts it after a crash or reboot and keeps logs. It is the Node counterpart of systemd + gunicorn supervising uvicorn.
- **Docker** packages Node, the built code and its `node_modules` into one image, as you may have done for a FastAPI app.
- One Node process serves many simultaneous users thanks to the event loop (section 4); FastAPI usually runs several uvicorn workers instead. If NeuroMEP ever runs several API processes, the in-memory login limiter would have to move to MongoDB or Redis so all processes share the counts.

Nginx in front

Nginx terminates HTTPS (a Let's Encrypt certificate), serves the web files and forwards `/api` to Node. Because React Router handles URLs such as `/projects/abc/floor` in the browser, Nginx must answer every unknown path with `index.html`. An illustrative configuration, not yet in the repository:

```
server {
  listen 443 ssl;
  server_name neuromep.example.com;              # real domain decided in WBS
1.8
  root /srv/neuromep/web;                        # contents of apps/web/dist

  location /api/ {
    proxy_pass http://127.0.0.1:4000;            # the Node API
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    client_max_body_size 20m;                    # plan PDFs up to 20 MB (A-
02)
  }
  location / {
    try_files $uri /index.html;                  # let React Router handle
page URLs
  }
}
```

Continuous integration

`.github/workflows/ci.yml` runs on every push to GitHub, in three parallel jobs:

1. **Build and test:** `npm ci`, format check, lint, typecheck, tests with coverage, build. The coverage report is kept as a download.
2. **Dependency audit:** `npm audit --audit-level=high` fails the run on a high-risk advisory.
3. **Code analysis:** Semgrep scans the code with JavaScript, TypeScript, Node.js and React security rule sets.

The Python equivalent would be a workflow running ruff, mypy, pytest-cov, pip-audit and bandit.

Backups

The design calls for a nightly backup of MongoDB (`mongodump`, the counterpart of `pg_dump`) and of the plan files on disk, set up with the VPS in WBS 1.8.

14. Concepts coming in later WBS steps

The rest of the schedule (26 Oct to 10 Nov 2026) brings in a handful of new ideas. Each is listed with the WBS item that needs it and the closest thing in the Python world.

WBS	What gets built	New concept or library	Closest Python-world idea
1.2.5	Autosave every 10 s, version locks, audit log	timers, optimistic locking (already in place)	background task + row version column
1.2.7	Plan PDFs on server disk, browser cache of the open and previous floor	file streams, the browser Cache API / IndexedDB	`aiofiles`, `FileResponse`
1.3.1	Component library with legend symbols, drag-and-drop, auto-tags	SVG symbols, HTML drag-and-drop, **Konva** canvas	no direct equivalent (UI)
1.3.2	Cable drawing with ports and orthogonal routing; length = route x scale + 1 m	canvas hit-testing, geometry in metres via calibration	Shapely-style geometry
1.3.4	DB loads, Excel load upload, live load table	reading `.xlsx` in the browser, derived state	pandas / openpyxl
1.4.x	Building view, risers, network validation, scenarios	graph traversal over the network model	networkx
1.5.1	Right-panel results, "Out of date" state	client state store (**Zustand**)	a shared state object
1.5.2	Frontend smoke tests	**Playwright**	pytest-playwright
1.6.x	Calculation engine C1 to C12	pure functions, property-style tests, 90 % coverage	NumPy-free pure Python modules + pytest
1.6.8	Independent checker compared on random networks	**a separate Python implementation** that must agree with the TypeScript engine	your home ground
1.7.1	Engine behind the Cable Size button	**Web Worker** + `postMessage`	`ProcessPoolExecutor`

WBS	What gets built	New concept or library	Closest Python-world idea
1.7.2 / 1.7.3	Excel schedules and calculation sheet, never overwritten (`_v1` , `_v2`)	**ExcelJS**, browser downloads	openpyxl / XlsxWriter
1.8.x	Staging, security scan, UAT	Nginx, PM2, OWASP ZAP, `mongodump`	gunicorn + systemd, `pg_dump`

Canvas drawing with Konva

The plan viewer today draws the PDF onto an HTML `<canvas>` with PDF.js and handles zoom, pan and calibration clicks itself (`apps/web/src/viewer/`). Placing components and drawing cables needs objects that can be dragged, selected and hit-tested. **Konva** gives the canvas a tree of shapes (layers, groups, symbols, lines) with events, like a small scene graph. Every position is stored in **metres** in the model and converted to screen pixels through the calibration, so zooming never changes the data.

Web Workers: keeping the screen responsive

A Web Worker is a second JavaScript thread in the browser with no access to the page. The app sends it the network model with `postMessage` ; the worker runs the engine and posts the results back. The canvas keeps responding while a large building is sized. The same engine code also runs in the API (before exports), which is why it lives in its own package with no browser or server dependencies.

A client state store

The open project now lives in React context and the reducer. Floor Setup will hold many fast-changing things (selection, drag position, results per element). **Zustand** is a small store that components subscribe to selectively, so moving one symbol does not redraw the whole page. It is the frontend counterpart of keeping shared state in one well-defined object.

Real-time collaboration (design doc Section 8, after this schedule)

Several engineers working on one building at once needs pushes from the server: **WebSockets** through **Socket.IO** (FastAPI has WebSocket routes; python-socketio is the direct port). On the server, MongoDB **change streams** report every change to a collection as it happens, like PostgreSQL `LISTEN/NOTIFY` . Element locks and discipline greying (D-16) are built on these.

Single sign-on later

When MAEC One (OI-28) is decided it will most likely be **OpenID Connect**: the browser is redirected to the company login, which sends back a code the API exchanges for the user's identity (FastAPI users know this from Authlib). Only a new `AuthProvider` and two routes are needed; sessions and cookies stay as they are (section 9).

The Revit add-in

`revit-addin/` will hold a C# .NET plug-in for Autodesk Revit (#TASK-33). It will talk to the same HTTP API, and floors will gain a second plan kind, `kind: 'revit'`, next to `kind: 'pdf'`.

15. Directory and file walkthrough

Every file in the repository as of commit `f8a0757` (28 Oct 2026), grouped by package. Test files (`*.test.ts`, `*.test.tsx`) sit next to the file they test and are left out of the tables. In a FastAPI project you would have roughly: `app/main.py`, `app/routers/`, `app/models/`, `app/schemas/`, `app/db.py`, `alembic/`, `tests/` and a separate `frontend/`.

Repository root

Path	Purpose
`package.json`	workspaces list and root commands (`dev`, `test`, `lint`, `build` ...)
`package-lock.json`	exact versions of every library (never edited by hand)
`tsconfig.base.json`	TypeScript settings shared by all packages (strict mode)
`eslint.config.js`	lint and security rules
`.prettierrc.json`, `.prettierignore`	formatting style
`vitest.config.ts`, `vitest.shared.ts`	test runner: five packages as projects; tests read TypeScript source
`.env.example`	settings template; copy to `.env` (never committed)
`.nvmrc`, `.editorconfig`, `.gitattributes`, `.gitignore`, `.semgrepignore`	Node version, editor, line endings, ignored files, scanner exclusions

Path	Purpose
`.github/workflows/ci.yml`	CI: tests, audit, Semgrep on every push
`CLAUDE.md`	standing project rules and decisions for the AI assistant
`docs/`	design document (`NeuroMEP_SRS_SDD.md`), `CHANGELOG.md` , CableSizer source rules, the Word design doc and WBS Gantt, figures
`NeuroMEP UI/`	the 14 Figma screens the UI follows
`prompts/`	the original task prompts
`revit-addin/`	future C# Revit plug-in (README only)

packages/shared: data shapes and rules for everyone

File	Purpose
`src/index.ts`	re-exports the package (`import { Project } from '@neuromep/shared'`)
`src/roles.ts`	the six roles and their labels
`src/users.ts`	`UserSummary` and helpers (initials for the avatar)
`src/access.ts`	who may see, create or edit a project; the project-card summary
`src/errors/catalogue.ts` , `index.ts`	every error code with message, severity and action
`src/project/schema.ts`	`ProjectSchema` : the whole project document
`src/project/enums.ts`	fixed lists: design stages, categories, regions, load types, page keys, paper sizes
`src/project/factory.ts`	`createEmptyProject()`
`src/project/readiness.ts`	Import File validation (`page1Issues`) and input limits
`src/project/floors.ts`	floor list from No of Floors / No of Basements; floor names

File	Purpose
`src/project/calibration.ts`	plan-scale maths (metres per PDF unit)
`src/project/demand.ts`	demand-factor helpers (TBC means 1.0, result Provisional)
`src/project/progress.ts`	tab order: furthest page reached (A-12)
`src/project/file.ts`	project `.json` file: write and read with friendly errors
`src/project/migrate.ts`	upgrades older stored projects on read
`src/model/elements.ts`, `cable.ts`, `results.ts`, `index.ts`	network model: components, cables, Cable Size results and checks
`src/testing.ts`	test fixtures such as `completeProject()`

packages/rulepacks: reference data

File	Purpose
`dubai/v1.json`	Dubai rule pack: calculation defaults (TBC values are `null`)
`cablesizer/lookup-tables.json`	CableSizer tables as typed rows (archive, results stay Provisional)
`scripts/convert-cablesizer.ts`	one-time converter from `docs/reference/cablesizer_lookup_rules.json`
`src/schema.ts`	rule-pack shape (Zod)
`src/registry.ts`	available rule packs; active regions (Dubai only)
`src/apply.ts`	pre-fills a project from a rule pack without overwriting user values
`src/cablesizer/schema.ts`, `lookup.ts`, `index.ts`	typed table lookups

packages/engine: calculation engine

File	Purpose
`src/index.ts`	placeholder (`ENGINE_VERSION`) until WBS 1.6

File	Purpose
`src/boundary.test.ts`	guards the rule that the engine has no UI, database or server dependencies

apps/api: the backend

File	Purpose	FastAPI counterpart
`src/server.ts`	start-up: config, database, user file, sessions, listen on 4000	`main.py` + `uvicorn.run`
`src/config.ts`	environment settings, validated	`pydantic-settings`
`src/app.ts`	all routes	`routers/*.py`
`src/errors.ts`	`sendError` with catalogue codes	exception handlers
`src/auth/provider.ts`	`AuthProvider` interface (swap point for MAEC One)	an abstract base class
`src/auth/config-provider.ts`	users from the user file	a user repository
`src/auth/passwords.ts`	scrypt hashing	`passlib`
`src/auth/hash-password.ts`	`npm run hash-password` tool	a management command
`src/auth/sessions.ts`	session store (MongoDB with TTL; in-memory for tests)	a sessions table
`src/auth/middleware.ts`	cookie reading, `requireUser`, `currentUser`	`Depends(get_current_user)`
`src/auth/attempts.ts`	login attempt limit	`slowapi`
`config/users.dev.json`	development users with hashed passwords	seed data

File	Purpose	FastAPI counterpart
`src/db/mongo.ts`	MongoDB connection; built-in replica set for development	`database.py`
`src/projects/repository.ts`	project load/save across collections, transactions, optimistic locking	a repository / CRUD module
`src/plans/storage.ts`	floor-plan PDF storage (GridFS today)	file storage service

apps/web: the frontend

File or folder	Purpose
`index.html`, `src/main.tsx`	the HTML shell and the start-up code
`src/App.tsx`	route table
`src/api/client.ts`	`ApiClient`: one method per endpoint
`src/auth/AuthProvider.ts`, `AuthContext.tsx`, `ProtectedRoute.tsx`	sign-in/out, current user, redirect to Login
`src/layout/Shell.tsx`, `Toolbar.tsx`, `icons.tsx`, `shell.css`	page frame, top tabs, shared toolbar, icons
`src/workspace/ProjectWorkspace.tsx`	open-project context, Back/Next and tab rules
`src/workspace/useProjectEditor.ts`	load, edit, autosave, conflicts
`src/workspace/projectReducer.ts`, `formProps.ts`	every project change as an action; form helpers
`src/pages/LoginPage.tsx`, `login.css`	Login (Figma 0)
`src/pages/projects/`	All Projects (Figma 1)
`src/pages/import/`	Import File page and form (Figma 2)
`src/pages/floor/`	Floor Setup, Building tab, Demand Factors dialog (Figma 3.1 to 3.8)
`src/pages/schematic/`, `src/pages/output/`	Schematic and Output with sample content (Figma 4, 5)

File or folder	Purpose
`src/viewer/`	PDF viewer: `PlanViewer.tsx`, `pdf.ts`, `transform.ts` (screen to PDF coordinates), `snap.ts` (snap to line ends)
`src/components/`	reusable inputs, modal, engineer multi-select
`src/sample/sampleData.ts`	placeholder content until WBS 1.5.3 and 1.7
`src/styles/global.css`	colours, dark theme, common styles
`src/test-setup.ts`	test helpers loaded before screen tests
`vite.config.ts`	dev server, `/api` proxy, test settings

16. Cheat sheet and glossary

Python to TypeScript in one table

You would write in Python	Write in NeuroMEP
`len(items)`	`items.length`
`x in items`	`items.includes(x)`
`key in d`	`key in obj` or `map.has(key)`
`d.get(k, default)`	`obj[k] ?? fallback`
`sorted(items, key=lambda f: f.order)`	`[...items].sort((a, b) => a.order - b.order)` (sort changes the array, so copy first)
`any(...)` / `all(...)`	`items.some(...)` / `items.every(...)`
`next((f for f in floors if f.id == i), None)`	`floors.find((f) => f.id === i) ?? null`
`{**a, 'x': 1}`	`{ ...a, x: 1 }`
`', '.join(names)`	`names.join(', ')`
`str(n)` / `int(s)` / `float(s)`	`String(n)` / `Number.parseInt(s, 10)` / `Number(s)`

You would write in Python	Write in NeuroMEP
`isinstance(x, str)`	`typeof x === 'string'`
`datetime.now(timezone.utc).isoformat()`	`new Date().toISOString()`
`uuid.uuid4()`	`crypto.randomUUID()`
`json.dumps(obj, indent=2)` / `json.loads(s)`	`JSON.stringify(obj, null, 2)` / `JSON.parse(s)`
`print(...)` / `logging.info(...)`	`console.log(...)`
`raise ValueError('...')`	`throw new Error('...')`
`async def f(): ...` / `await f()`	`async function f() {}` / `await f()`
`asyncio.gather(a(), b())`	`Promise.all([a(), b()])`
`await asyncio.sleep(1)`	`await new Promise((r) => setTimeout(r, 1000))`
`@dataclass` / `BaseModel`	`interface` / `z.object(...)` + `z.infer`
`if __name__ == '__main__':`	a separate entry file (`server.ts` , `hash-password.ts`)

Glossary

Term	Meaning
BSON	MongoDB's binary storage form of JSON documents
bundle	the few optimised JavaScript files Vite builds from hundreds of source files
change stream	a live feed of changes from a MongoDB collection
collection	MongoDB's equivalent of a table
context (React)	data made available to every component below a provider
CORS	browser rules for requests to another origin; NeuroMEP avoids it with a proxy
CSRF	a forged request from another site using your cookies; blocked by `SameSite=Strict`

Term	Meaning
`dist/`	compiled output folder of a package
event loop	the single-threaded scheduler that runs async code (Node and asyncio)
hook	a React function starting with `use` that adds state, effects or context
httpOnly cookie	a cookie page JavaScript cannot read
JSX / TSX	HTML-like markup inside JavaScript / TypeScript files
middleware	an Express function `(req, res, next)` that runs before the route handler
monorepo / workspace	one repository with several npm packages linked together
optimistic locking	saving only if the `version` is still the one you loaded
Promise	a value that arrives later; awaited with `await`
reducer	a pure function (state, action) to new state
replica set	a group of MongoDB servers with copies of the data; needed for transactions
repository	the module that hides database code behind methods like `get` and `save`
session	the server's record that a browser is signed in
TTL index	an index that deletes documents when a date passes
`tsc`	the TypeScript compiler and type checker
Web Worker	a background thread in the browser
Zod schema	a runtime validator that also defines a TypeScript type